Taller 2

Sistema Avanzados De Producción

Natalia Alejandra Castilla - 45305

Karina Fernanda Paternina - 119600

Lina Sofia Garzón - 121025

Facultad De Ingeniería Industrial

Universidad ECCI

12 de Abril de 2026

INTRODUCCIÓN

En el presente trabajo se abordan problemas de regresión y clasificación mediante el uso de modelos de *machine learning*, haciendo especial énfasis en la implementación de redes neuronales como herramienta principal de análisis. El objetivo es recorrer el ciclo completo del aprendizaje automático, desde la comprensión del problema y el análisis de los datos, hasta la construcción, evaluación e interpretación de los modelos.

En la primera parte, correspondiente a la **Ruta A**, se desarrolla un problema de regresión enfocado en la predicción de un valor continuo. Para ello, se utilizan modelos como la regresión lineal y una red neuronal, evaluando su desempeño a través de métricas como MAE y RMSE. Posteriormente, en la **Ruta B**, se aborda un problema de clasificación binaria, comparando diferentes algoritmos como la regresión logística, el perceptrón y el modelo de *Random Forest*, con el fin de identificar cuál ofrece mejores resultados según las características del conjunto de datos.

Como caso de estudio, se trabaja con el problema de **Bike Sharing (UCI)**, cuyo objetivo es predecir la demanda de bicicletas en función de variables como condiciones climáticas y temporales. Este problema tiene una alta relevancia en el ámbito empresarial, ya que permite optimizar la toma de decisiones relacionadas con la planificación de la capacidad operativa, incluyendo la asignación de flota, la programación de turnos y el mantenimiento. Una predicción precisa de la demanda contribuye a evitar tanto la falta de disponibilidad del servicio como el uso ineficiente de los recursos.

En este contexto, el análisis desarrollado busca no solo construir modelos predictivos, sino también generar conocimiento útil para la toma de decisiones, integrando herramientas de regresión y clasificación que permitan mejorar la eficiencia y competitividad en entornos reales.

FASE NÚMERO 1: DESARROLLO ANALÍTICO: REGULARIZACIÓN RIDGE.

1. Representación Vectorial:

- $J(\beta) = \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^{p} \beta_j^2$

- $Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$ $\quad n \times 1$

- $X = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p} \\ 1 & x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & x_{n1} & x_{np} \end{bmatrix}$ $\quad (n \times (p+1))$

- $\beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix}$ $\quad ((p+1) \times 1)$

- $y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij}$

- $\sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2$

- $(Y - X\beta)^T (Y - X\beta)$

- $\lambda \sum_{j=1}^{p} \beta_j^2$

- $\sum_{j=1}^{p} \beta_j^2 = \beta^T \beta$

- $J(\beta) = (Y - X\beta)^T (Y - X\beta) + \lambda \beta^T I^* \beta$

Imagen 1

- $$I^* = \begin{bmatrix} 0 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{bmatrix}$$

- Sea $Y \in \mathbb{R}^{n \times 1}$ el vector de respuestas $X \in \mathbb{R}^{n \times (p+1)}$ la matriz de diseño que incluye una columna de unos para el intercepto, y $\beta \in \mathbb{R}^{(p+1) \times 1}$ el vector de parámetros.

- $y - X\beta$

- $(y - X\beta)^T (y - X\beta)$

- $J(\beta) = (y - X\beta)^T (y - X\beta) + \lambda \beta^T I^* \beta$

Imagen 2

2. Expansión de la Forma Cuadrática:

- $(Y - X\beta)^T (Y - X\beta)$

- $= Y^T Y - Y^T X\beta - (X\beta)^T Y + (X\beta)^T (X\beta)$

- $(X\beta)^T = \beta^T X^T$

- $(Y - X\beta)^T (Y - X\beta) = Y^T Y - Y^T X\beta - \beta^T X^T Y + \beta^T X^T X\beta$

- $Y^T X\beta = \beta^T X^T Y$

- $-Y^T X\beta - \beta^T X^T Y = -2\beta^T X^T Y$

- $(Y - X\beta)^T (Y - X\beta) = Y^T Y - 2\beta^T X^T Y + \beta^T X^T X\beta$

- $J(\beta) = Y^T Y - 2\beta^T X^T Y + \beta^T X^T X\beta + \lambda \beta^T \beta$

Imagen 3

- $(Y-X\beta)^T(Y-X\beta)$
- $(Y-X\beta)^T(Y-X\beta) = Y^TY - 2\beta^TX^TY + \beta^TX^TX\beta$
- $J(\beta) = Y^TY - 2\beta^TX^TY + \beta^TX^TX\beta + \lambda\beta^T\beta$

Imagen 4

3. Condición de Primer Orden:

- $J(\beta) = Y^TY - 2\beta^TX^TY + \beta^TX^TX\beta + \lambda\beta^T\beta$
- $\frac{\partial}{\partial\beta}(Y^TY) = 0$
- $\frac{\partial}{\partial\beta}(-2\beta^TX^TY) = -2X^TY$
- $\frac{\partial}{\partial\beta}(\beta^TA\beta) = 2A\beta$
- $\frac{\partial}{\partial\beta}(\beta^TX^TX\beta) = 2X^TX\beta$
- $\frac{\partial}{\partial\beta}(\lambda\beta^T\beta) = 2\lambda\beta$
- $\frac{\partial J(\beta)}{\partial\beta} = -2X^TY + 2X^TX\beta + 2\lambda\beta$
- $\frac{\partial J(\beta)}{\partial\beta} = 2(X^TX\beta + \lambda\beta - X^TY)$
- $\frac{\partial J(\beta)}{\partial\beta} = 0$

Imagen 5

- $-2X^TY + 2X^TX\beta + 2\lambda\beta = 0$
- $X^TX\beta + \lambda\beta = X^TY$
- $(X^TX + \lambda I)\beta = X^TY$
- $J(\beta) = Y^TY - 2\beta^TX^TY + \beta^TX^TX\beta + \lambda\beta^T\beta$
- $\frac{\partial}{\partial\beta}(-2\beta^TX^TY) = -2X^TY$
- $\frac{\partial}{\partial\beta}(\beta^TX^TX\beta) = 2X^TX\beta$
- $\frac{\partial}{\partial\beta}(\lambda\beta^T\beta) = 2\lambda\beta$

- $\frac{\partial J(\beta)}{\partial\beta} = -2X^TY + 2X^TX\beta + 2\lambda\beta$
- $-2X^TY + 2X^TX\beta + 2\lambda\beta = 0$
- $(X^TX + \lambda I)\beta = X^TY$
- $\hat{\beta}_{ridge} = (X^TX + \lambda I)^{-1}X^TY$

Imagen 6

4. Solución Cerrada:

- $(X^TX + \lambda I)\beta = X^TY$
- $(X^TX + \lambda I)^{-1}(X^TX + \lambda I)\beta = (X^TX + \lambda I)^{-1}X^TY$
- $(X^TX + \lambda I)^{-1}(X^TX + \lambda I) = I$
- $I\beta = (X^TX + \lambda I)^{-1}X^TY$
- $\beta = (X^TX + \lambda I)^{-1}X^TY$
- $\hat{\beta}_{ridge} = (X^TX + \lambda I)^{-1}X^TY$
- $(X^TX + \lambda I)\beta = X^TY$
- $(X^TX + \lambda I)^{-1}(X^TX + \lambda I)\beta = (X^TX + \lambda I)^{-1}X^TY$
- $\beta = (X^TX + \lambda I)^{-1}X^TY$
- $\hat{\beta}_{ridge} = (X^TX + \lambda I)^{-1}X^TY$

Imagen 7

PREGUNTAS FASE 1:

1. Sobre la Invertibilidad:

La matriz **X T X** puede no ser invertible cuando existe una alta correlación entre las variables (multicolinealidad) o cuando hay más variables que datos. En estos casos, la matriz se vuelve singular y no se puede invertir.

Sin embargo, al agregar el término de regularización **λI**, donde **λ > 0** , se logra que la matriz:

X T X + λI

Sea invertible.

Esto ocurre porque la suma de λI incrementa los valores propios de la matriz, haciendo que todos sean positivos. Como resultado, la matriz se vuelve definida positiva y, por lo tanto, siempre tiene inversa.

2. Efecto del Parámetro λ:

El parámetro **λ** controla el nivel de regularización del modelo, es decir, qué tanto se penalizan los coeficientes.

Cuando $\lambda \to 0$, el término de regularización prácticamente desaparece, por lo que el modelo se comporta como una regresión por Mínimos Cuadrados Ordinarios (OLS). En este caso, los coeficientes pueden tomar valores grandes, el modelo se ajusta muy bien a los datos de entrenamiento y tiene **bajo sesgo pero alta varianza**, lo que puede llevar a sobreajuste

Por el contrario, cuando $\lambda \to \infty$, la penalización es muy fuerte, por lo que los coeficientes tienden a acercarse a cero. Esto simplifica mucho el modelo, reduciendo su complejidad. En este caso, el modelo tiene **alto sesgo pero baja varianza**, lo que puede generar subajuste.

En resumen, **λ** permite controlar el equilibrio entre sesgo y varianza: valores pequeños generan modelos más complejos y sensibles a los datos, mientras que valores grandes producen modelos más simples y estables.

FASE NÚMERO 2 - OPTIMIZACIÓN NUMÉRICA Y DESCENSO DE GRADIENTE

Semana 6: Descenso de Gradiente

1.Impacto de la Tasa de Aprendizaje (n):

Gráfica Actual

- - - Configuración De La Simulación - - -

* Start_Point=(0.0,0.1) *Punto de Inicio
*Learning_Grate =**0.001** *Tasa de Aprendizaje (eta)

*Num_Iterations =1000 *# Número de pasos

a. Configuración inicial del algoritmo

El código implementa el algoritmo de descenso de gradiente con una configuración base que permite analizar su comportamiento inicial. Se define un punto de inicio cercano al origen (start point = **[0.0, 0.1]),** desde el cual el algoritmo comienza la búsqueda del mínimo.

La tasa de aprendizaje se establece en **0.0001,** lo que implica que el algoritmo realiza pasos pequeños en cada iteración, favoreciendo una convergencia estable pero lenta. Además, se utilizan **1000** iteraciones, permitiendo que el modelo actualice progresivamente su posición.

El algoritmo se ejecuta mediante la función gradient descent, generando una trayectoria que muestra cómo el punto se desplaza hacia el mínimo de la función.

- Descripción de las Gráficas

La gráfica 3D muestra la superficie de la función y cómo el algoritmo desciende desde el punto inicial hacia una zona de menor valor.

La gráfica de curvas de nivel permite ver el recorrido del algoritmo en dos dimensiones, donde se evidencia cómo sigue la dirección de descenso hasta acercarse a un mínimo.

En conjunto, las gráficas muestran el proceso de convergencia del algoritmo de descenso de gradiente.

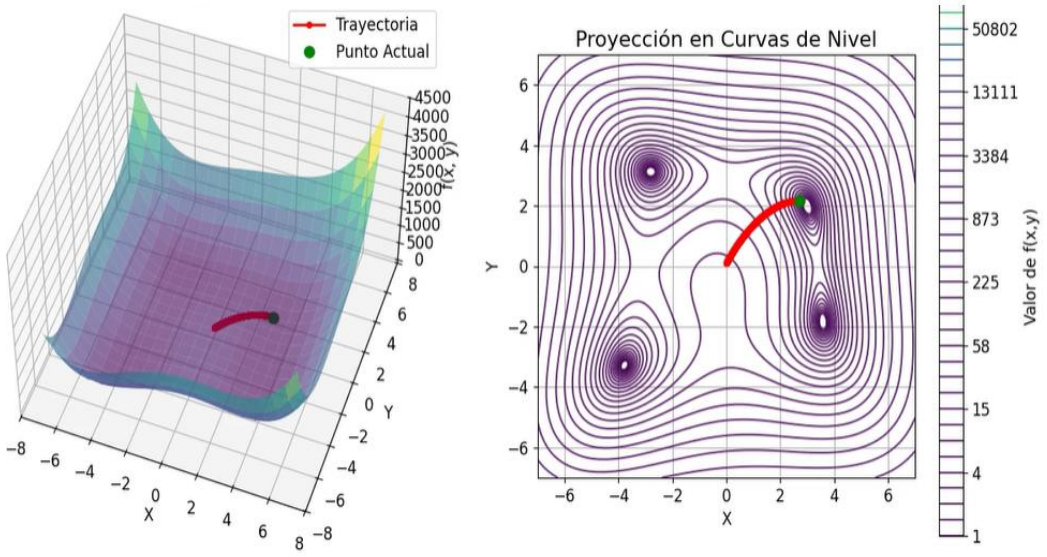


Imagen.8 Proyección Curvas de nivel

- - - Configuración De La Simulación - - -

* Start_Point=(0.0,0.1) *Punto de Inicio
*Learning_Grate =**0.5** *Tasa de Aprendizaje (eta)
*Num_Iterations =1000 *# Número de pasos

b. Incremente el aprendizaje a un valor significativamente más alto (ej. 0.5)

- **¿Qué sucede con la trayectoria? ¿Logra llegar al centro de las curvas de nivel o comienza oscilar/divergir?**

Al incrementar la tasa de aprendizaje a un valor alto como 0.5, se observa que el algoritmo deja de comportarse de manera estable. En lugar de seguir una trayectoria suave hacia el mínimo, el punto realiza saltos muy grandes dentro de la superficie.

Esto provoca que el algoritmo no logre converger al centro de las curvas de nivel, sino que comienza a oscilar e incluso a divergir, alejándose de la solución óptima.

Este comportamiento se debe a que el tamaño del paso es demasiado grande en relación con la pendiente de la función, lo que hace que el algoritmo "se pase" del mínimo en cada iteración, generando inestabilidad.

- **Descripción de Gráfico**

La gráfica 3D muestra que la trayectoria del algoritmo presenta desplazamientos abruptos, alejándose de la zona de mínimo en lugar de descender progresivamente. Esto evidencia que el algoritmo pierde estabilidad.

En la gráfica de curvas de nivel, se observa que la trayectoria no sigue un camino ordenado hacia el centro, sino que realiza saltos amplios entre regiones, lo que indica un comportamiento oscilante.

En conjunto, las gráficas demuestran que una tasa de aprendizaje alta impide la convergencia del algoritmo, generando movimientos inestables y evitando que alcance el mínimo de la función.

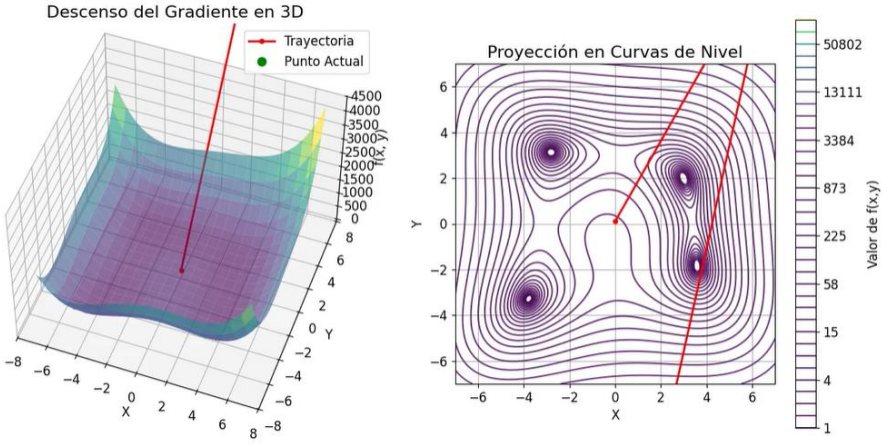


Imagen.9 Descenso de Gradiente 3D/Proyección en curvas de nivel

c. Reduce el aprendizaje Rate a un valor extremadamente pequeño 10-6 observe el resultado tras 1000 iteraciones. ¿Es suficiente este tiempo de cómputo para alcanzar el mínimo?

Al utilizar una tasa de aprendizaje extremadamente baja (10^{-6}), el algoritmo se comporta de manera estable, pero avanza muy lentamente. Como resultado, después de 1000 iteraciones no logra alcanzar el mínimo, ya que los pasos son demasiado pequeños

- Descripción de la Gráfica.

Las gráficas muestran un desplazamiento muy corto desde el punto inicial. La trayectoria no alcanza el centro de las curvas de nivel, lo que evidencia que el algoritmo converge de forma lenta y no llega al mínimo en el tiempo establecido.

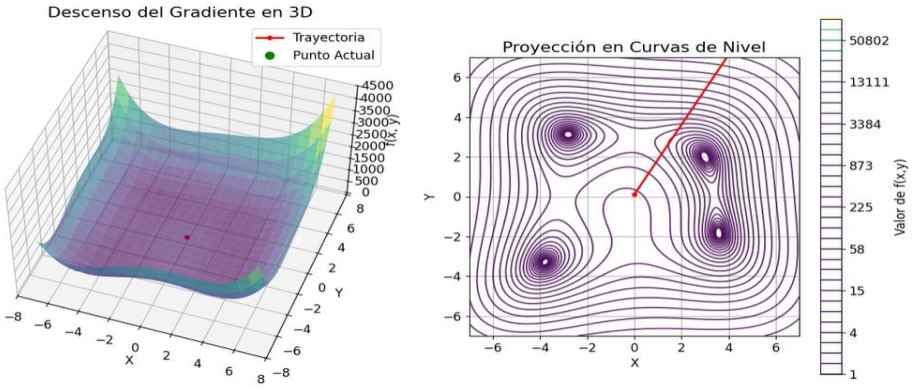


Imagen.10 Descenso de Gradiente 3D

d. Modifique el start point a coordenadas alejadas del origen, por ejemplo [6.0, -6.0] o [-5.0, 5.0].

¿El algoritmo llega siempre al mismo punto final? Explique si la topografía de la función f(X, Y) sugiere la presencia de múltiples valles o mínimos locales.

Al modificar el punto inicial a una coordenada alejada del origen, en este caso [6.0, -6.0], se observa que el algoritmo de descenso de gradiente sigue una trayectoria diferente a la obtenida en la configuración base. En lugar de dirigirse hacia la región central, el recorrido se orienta hacia el valle ubicado en la zona inferior derecha de la superficie.

Esto muestra que el algoritmo no siempre llega al mismo punto final, ya que el resultado depende del punto desde el cual inicia la búsqueda. La trayectoria está determinada por la pendiente local de la función en la zona inicial, por lo que el algoritmo tiende a converger hacia el mínimo más cercano dentro de esa región.

La topografía de la función f(X,Y), observada en la gráfica de curvas de nivel, sugiere la presencia de múltiples valles o mínimos locales. Esto se evidencia en las distintas regiones cerradas de las curvas de nivel, donde cada una representa zonas de atracción diferentes. Por esta razón, cambiar el punto inicial puede llevar al algoritmo a converger en un mínimo distinto.

– Descripción del Gráfico

La gráfica muestra que, al iniciar en **[6.0, -6.0]**, el algoritmo desciende hacia el valle más cercano ubicado en la parte

inferior derecha. La trayectoria no se dirige al centro de la función, sino a un mínimo local de esa región, lo que confirma que el punto inicial influye directamente en el resultado final.

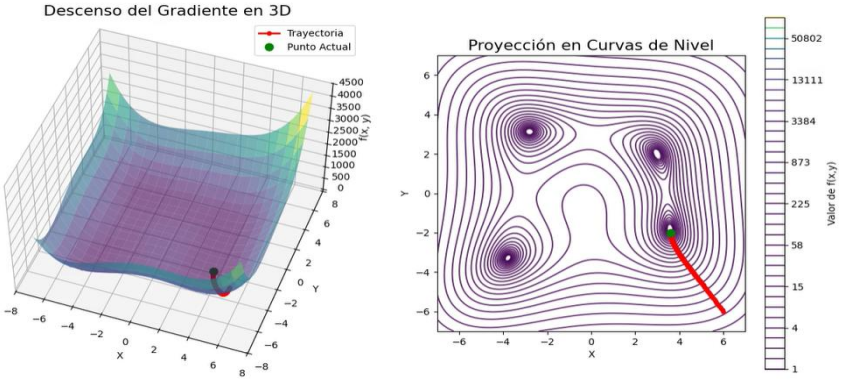


Imagen.11 Descenso de Gradiente 3D

- **PREGUNTAS FASE 2**

1. **El Fenómeno de "Overshooting": Explique físicamente que ocurre cuando el paso (η·∇f) es más grande que la distancia al m´ınimo. ¿Cómo se visualiza esto en el gráfico de curvas de nivel?**

El fenómeno de overshooting ocurre cuando el tamaño del paso (η·∇f)(\eta \cdot \nabla f)(η· ∇f) es mayor que la distancia que existe entre el punto actual y el mínimo. En términos físicos, esto significa que el algoritmo avanza "demasiado", sobrepasando el punto óptimo en lugar de acercarse a él de manera progresiva.

En lugar de detenerse en el mínimo, el algoritmo cruza hacia el otro lado y, en la siguiente iteración, vuelve a sobrepasar en sentido contrario. Esto genera un comportamiento de oscilación alrededor del mínimo, ya que el algoritmo no logra estabilizarse.

En el gráfico de curvas de nivel, este fenómeno se visualiza como una trayectoria que cruza repetidamente las curvas hacia lados opuestos del centro, formando un patrón de "zigzag" en lugar de una aproximación suave hacia el mínimo.

2. **Eficiencia Computacional: Si el algoritmo se detiene en la iteración 1000 pero aún se encuentra lejos del mínimo, ¿qué estrategia es mejor: aumentar el número de iteraciones o ajustar la tasa de aprendizaje? Justifique su respuesta.**

Si después de 1000 iteraciones el algoritmo aún se encuentra lejos del mínimo, resulta más eficiente ajustar la tasa de aprendizaje en lugar de simplemente aumentar el número de iteraciones.

Esto se debe a que, si el learning rate es demasiado pequeño, el algoritmo avanzará muy lentamente, por lo que aumentar las iteraciones solo incrementa el tiempo de cómputo sin garantizar una mejora significativa en el resultado. En cambio, al ajustar la tasa de aprendizaje a un valor más adecuado, se pueden lograr avances más significativos en cada iteración.

Por lo tanto, la estrategia más efectiva es encontrar un equilibrio en la tasa de aprendizaje que permita una convergencia más rápida sin perder estabilidad, en lugar de depender únicamente de un mayor número de iteraciones.

3. **Generalización: En un problema de ingeniería con cientos de variables (como la optimización de una cadena de suministro), ¿por qué es peligroso confiar en un único punto de inicio aleatorio?**

En problemas de ingeniería con muchas variables, como la optimización de una cadena de suministro, es riesgoso confiar en un único punto de inicio aleatorio. Esto se debe a que funciones más complejas suelen presentar múltiples mínimos locales, y el algoritmo puede quedar atrapado en uno de ellos dependiendo del punto inicial.

Si solo se utiliza un punto de inicio, no hay garantía de que la solución encontrada sea la mejor posible (mínimo global), lo que puede llevar a decisiones subóptimas en el contexto real del problema.

Por esta razón, es recomendable utilizar múltiples puntos de inicio o estrategias adicionales que permitan explorar mejor el espacio de soluciones, reduciendo el riesgo de quedarse en un mínimo local y aumentando la probabilidad de encontrar una solución óptima.

FASE NÚMERO 3:REDES NEURONALES (MLP) Y AJUSTE DE HIPERPAR´AMETROS

- Ejecuta el modelo MLP Regressor inicial con la configuración base (hidden layer sizes=(16, 8, 4)). Observe los resultados de las m´etricas MAE, RMSE y R2

FASE A

1. **¿Cuál de las dos penaliza con mayor severidad los errores grandes (outliers) en las ventas y por qué?**

El modelo MLP Regressor, configurado con una arquitectura base de capas ocultas (16, 8, 4), presenta los siguientes resultados:

- **Conjunto de entrenamiento (Train):**
  MAE: 47.086
  RMSE: 68.547
  R^2: 0.858
- **Conjunto de prueba (Test):**
  MAE: 47.175
  RMSE: 69.458
  R^2: 0.848

El RMSE es la métrica que penaliza con mayor severidad los errores grandes (outliers). Esto se debe a que eleva los errores al cuadrado antes de promediarlos, lo que hace que los valores más altos tengan un impacto mucho mayor en el resultado final.

Esto se puede relacionar con las gráficas obtenidas. En la gráfica de y real vs y pred, se observa que existe mayor dispersión en los valores altos, lo que indica la presencia de errores más grandes en esas zonas. Además, el histograma de residuos muestra que, aunque la mayoría de los errores están cerca de cero, también existen algunos valores más alejados.

Debido a esta presencia de errores grandes, el RMSE los amplifica más que el MAE, lo que explica por qué su valor es mayor en los resultados obtenidos. En cambio, el MAE trata estos errores de manera más proporcional, sin darles un peso excesivo.

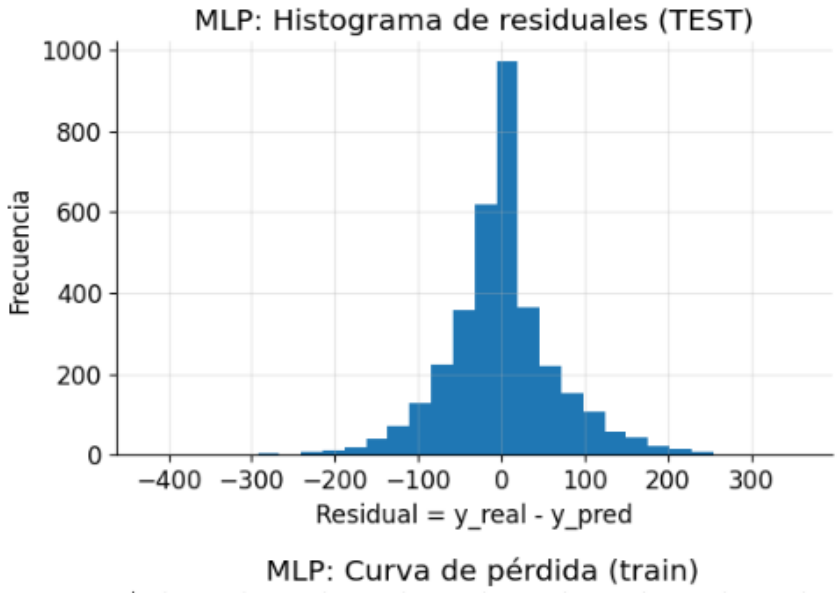


Imagen.12 Histograma de residuales (TEST)

2. ¿Qué fenómeno está ocurriendo? Proponga una solución basada en lo aprendido en la Fase 1 del taller.

De acuerdo con los resultados obtenidos, el valor de R 2 en entrenamiento (0.858) es muy cercano al valor en prueba (0.848), lo que indica que no se está presentando un sobreajuste significativo. El modelo logra generalizar de manera adecuada, ya que mantiene un rendimiento similar en ambos conjuntos.

Sin embargo, en el entrenamiento aparece una advertencia (*Convergence Warning*) que indica que el modelo no ha convergido completamente en el número máximo de iteraciones (200). Esto sugiere que el modelo aún no ha alcanzado su mejor ajuste posible.

Teniendo en cuenta lo aprendido en la Fase 1, una posible mejora es aplicar regularización mediante el parámetro *alpha*, lo cual ayudaría a controlar la complejidad del modelo. Adicionalmente, se podría aumentar el número de iteraciones (*maxiter*) para permitir que el modelo termine de converger.

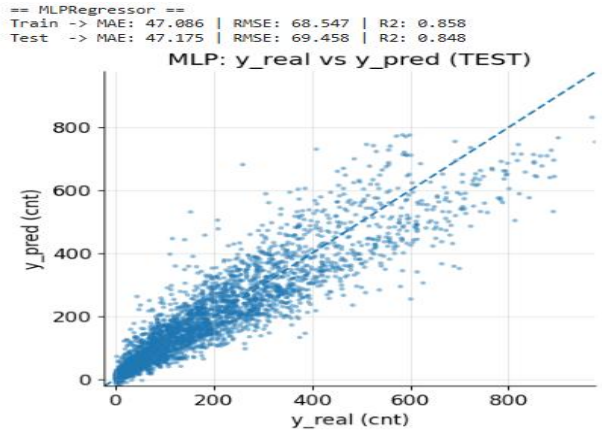


Imagen.13 MLP y real Vs Pred (TEST)

3. **Al observar la gráfica de y real vs y pred, ¿se identifica algún patrón de error sistemático (ej. subestimación en ventas altas)?**

Sí, al observar la gráfica de y real vs y pred, se puede identificar un leve patrón de error sistemático. En general, los puntos siguen la línea diagonal, lo que indica que el modelo logra captar bien la tendencia de los datos.

Sin embargo, a medida que los valores reales aumentan, se observa una mayor dispersión y varios puntos quedan por debajo de la línea ideal. Esto indica una ligera subestimación en las ventas más altas, ya que el modelo tiende a predecir valores menores que los reales en esos casos.

En conclusión, aunque el modelo presenta un buen ajuste general, existe un pequeño patrón de error en valores elevados, donde su precisión disminuye ligeramente.

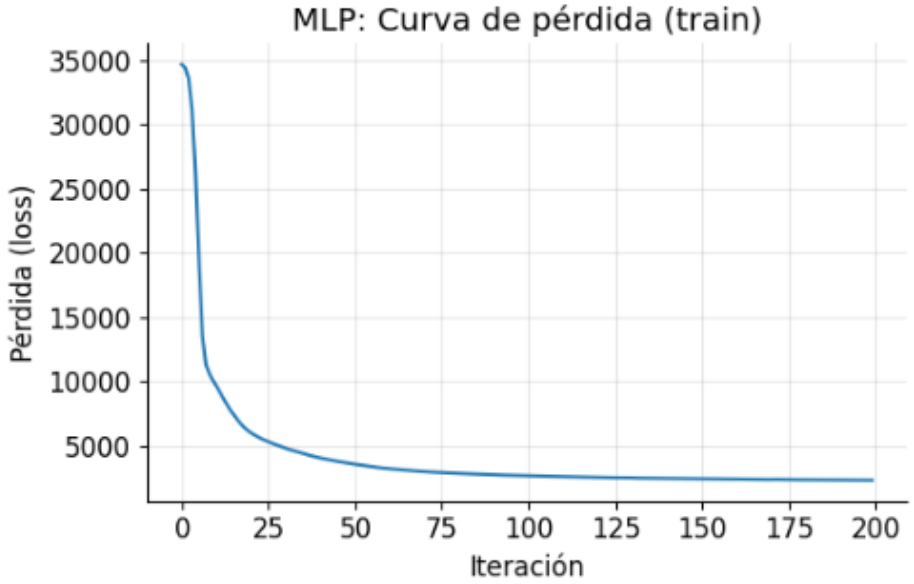


Imagen.14 MLP.Curva de Pérdida (Train)

FASE NUMERO 4: PROPAGACIÓN HACIA ADELANTE (FORWARD PROPAGATION) MANUAL

1. **Puntaje de la Capa Oculta (Z (1)):**

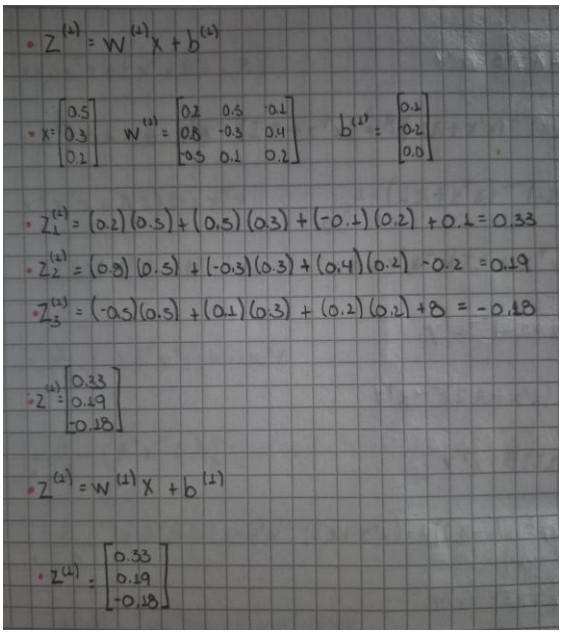


Imagen.15

2. Activación de la Capa Oculta (A(1)):

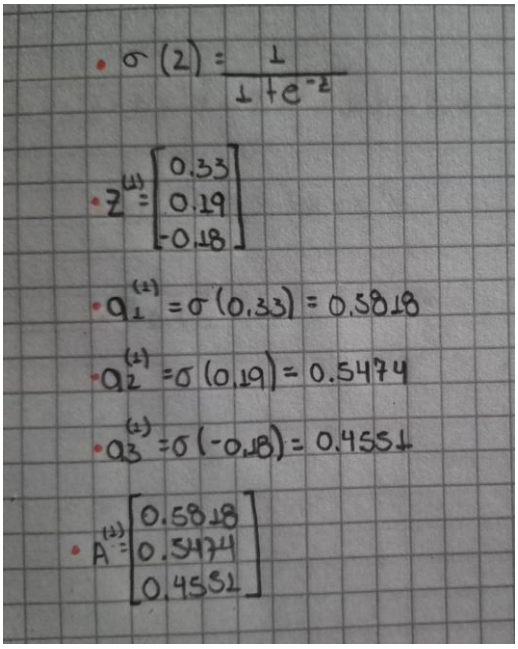
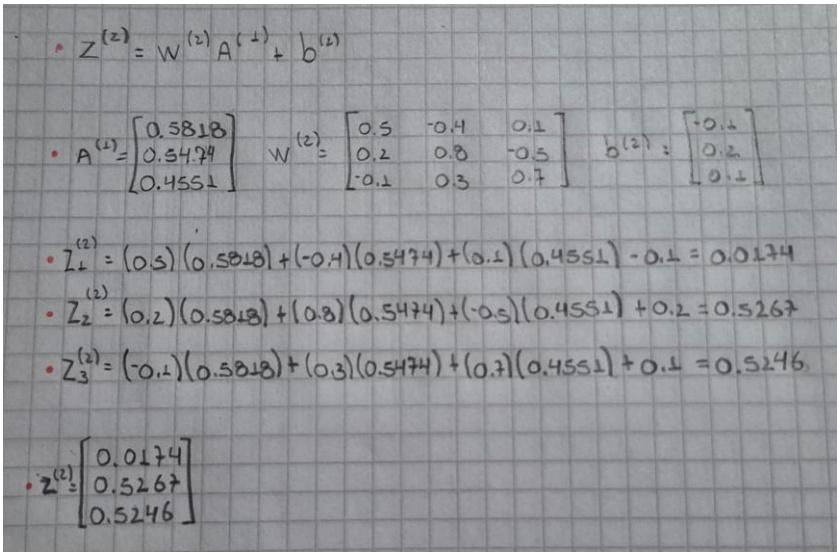


Imagen.16

3. Puntaje de la Capa de Salida (Z (2)):



- $Z^{(2)} = W^{(2)} A^{(1)} + b^{(2)}$

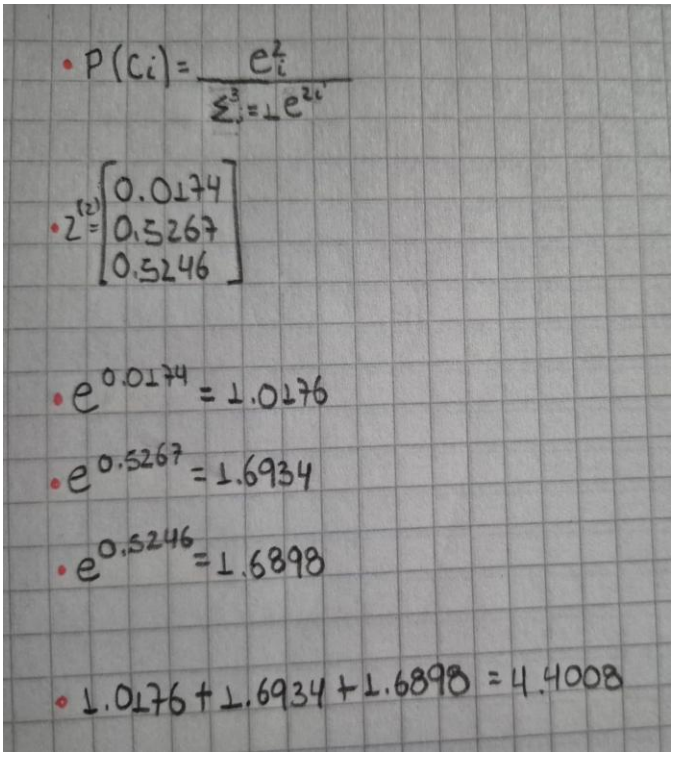
- $A^{(1)} = \begin{bmatrix} 0.5818 \\ 0.5474 \\ 0.4551 \end{bmatrix}$ $W^{(2)} = \begin{bmatrix} 0.5 & -0.4 & 0.1 \\ 0.2 & 0.8 & -0.5 \\ -0.1 & 0.3 & 0.7 \end{bmatrix}$ $b^{(2)} = \begin{bmatrix} -0.1 \\ 0.2 \\ 0.1 \end{bmatrix}$

- $Z_1^{(2)} = (0.5)(0.5818) + (-0.4)(0.5474) + (0.1)(0.4551) - 0.1 = 0.0174$
- $Z_2^{(2)} = (0.2)(0.5818) + (0.8)(0.5474) + (-0.5)(0.4551) + 0.2 = 0.5267$
- $Z_3^{(2)} = (-0.1)(0.5818) + (0.3)(0.5474) + (0.7)(0.4551) + 0.1 = 0.5246$

- $Z^{(2)} = \begin{bmatrix} 0.0174 \\ 0.5267 \\ 0.5246 \end{bmatrix}$

Imagen.17

4. Probabilidades de Clase (A(2)):



- $P(C_i) = \dfrac{e^{z_i}}{\sum_{i=1}^{3} e^{z_i}}$

- $Z^{(2)} = \begin{bmatrix} 0.0174 \\ 0.5267 \\ 0.5246 \end{bmatrix}$

- $e^{0.0174} = 1.0176$
- $e^{0.5267} = 1.6934$
- $e^{0.5246} = 1.6898$

- $1.0176 + 1.6934 + 1.6898 = 4.4008$

Imagen.18

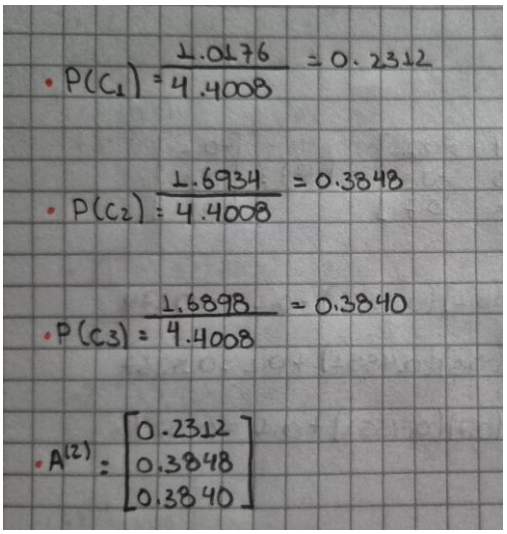


Imagen.19

PREGUNTAS FASE 4

1. Interpretación de Resultados:

Según las probabilidades obtenidas, la clase con mayor valor es C2 (Impacto Medio), con una probabilidad aproximada de 0.3848.

Por lo tanto, la red neuronal clasifica la campaña publicitaria como de Impacto Medio, ya que es la categoría más probable entre las tres opciones.

2. **Función de Activación:**

Se utiliza la función Softmax en la capa de salida porque permite transformar los valores obtenidos en probabilidades que suman 1.

A diferencia de una función lineal o una sigmoide simple, Softmax toma en cuenta todas las salidas al mismo tiempo y las normaliza, asegurando que cada valor representa la probabilidad de pertenecer a una clase específica.

Esto es fundamental en problemas de clasificación multiclase, ya que permite interpretar fácilmente los resultados y tomar una decisión basada en la mayor probabilidad.

3. Rol del Bias:

El bias permite ajustar el modelo de forma más flexible, desplazando la función de activación de cada neurona.

Si todos los bias fueran cero, la red perdería capacidad de representación, ya que todas las salidas dependerían únicamente de las combinaciones lineales de las entradas. Esto limitaría la capacidad del modelo para ajustarse correctamente a los datos.

En otras palabras, el bias ayuda a que la red pueda aprender patrones más complejos; sin él, el modelo sería más rígido y menos preciso.

Fase Numero 5: Evaluación de Modelos y Toma de Decisiones (Caso: Mantenimiento Predictivo)

En esta fase se evaluó el desempeño de una red neuronal aplicada a un problema de mantenimiento predictivo, cuyo objetivo es identificar si una máquina presenta una falla. Debido al desbalance de clases en el conjunto de datos, se implementó el uso de pesos **(class weight)** para mejorar la detección de la clase minoritaria. A partir de los resultados obtenidos, se realiza un análisis de métricas, costos y toma de decisiones desde una perspectiva de ingeniería.

Análisis de Datos

valores:

3. Peso clase 0: 0.5175
4. Peso clase 1: 14.7601
5. Accuracy: 0.9990
6. Recall clase 1: 0.9706

De la matriz de confusión:

- TN = 1932
- FP = 0
- FN = 2
- TP = 66

PREGUNTAS FASE 5

1. **Análisis de Pesos: Si el modelo indica que los errores en la Clase 1 (Fallo) pesarán aproximadamente 14.7 veces más que en la Clase 0, ¿qué intenta corregir el algoritmo al configurar el parámetro class weight durante el entrenamiento?**

Para abordar el desbalance de clases, se calcularon los pesos utilizando la función class weight, obteniendo valores aproximados de 0.52 para la Clase 0 (No falla) y 14.7 para la Clase 1 (Fallo). Esto implica que los errores cometidos al no detectar una falla real tienen un peso mucho mayor durante el entrenamiento del modelo.

En términos prácticos, esta configuración obliga al algoritmo a prestar mayor atención a los casos de fallas, evitando que el modelo se incline hacia la clase mayoritaria. De esta manera, se mejora la capacidad de detección de eventos críticos, lo cual es fundamental en un contexto de mantenimiento predictivo, donde los errores pueden generar altos costos operativos.

2. **La Trampa de la Exactitud: Explique porque en este caso un Accuracy del 95% podría considerarse un resultado deficiente para la gestión de mantenimiento si el modelo no logra detectar los fallos reales?**

En problemas con datos desbalanceados, una alta exactitud (accuracy) no necesariamente refleja un buen desempeño del modelo. Esto se debe a que el modelo podría acertar la mayoría de los casos simplemente prediciendo la clase mayoritaria, sin identificar correctamente los eventos más importantes.

En este caso, aunque el modelo presenta una exactitud cercana al 99.9%, esto por sí solo no garantiza una buena solución. En el contexto de mantenimiento predictivo, lo más relevante es detectar las fallas reales, ya que estas tienen un impacto económico significativo. Por lo tanto, métricas como el recall resultan más apropiadas para evaluar el desempeño del modelo.

- Matriz de Confusión

A partir de la matriz de confusión obtenida, se identificaron los siguientes valores:

- TN (Verdaderos Negativos): 1932
- FP (Falsos Positivos): 0
- FN (Falsos Negativos): 2
- TP (Verdaderos Positivos): 66

Estos resultados indican que el modelo clasifica correctamente todos los casos de "No falla" y detecta la gran mayoría de las fallas reales, con únicamente dos casos no identificados. Además, no se presentan falsas alarmas, lo cual reduce costos innecesarios en inspecciones.

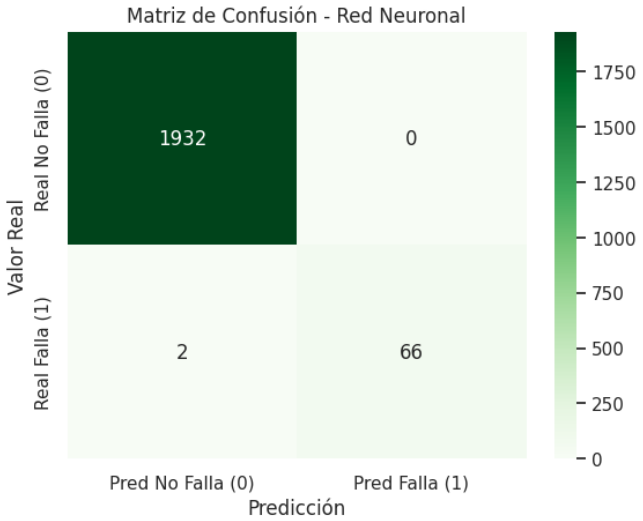


Imagen.20 Matriz de Confusión - Red Neuronal

- **Cálculo de Métricas**

- Precisión

-

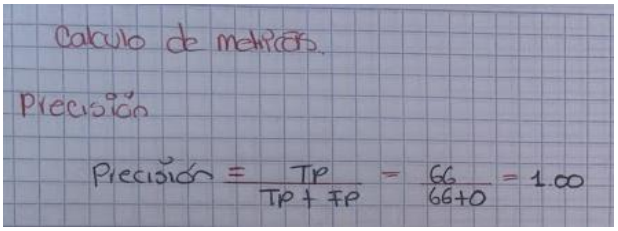


Imagen.21

La precisión del modelo es del **100%,** lo que indica que todas las alertas de falla generadas fueron correctas, sin presencia de falsas alarmas.

- Recall

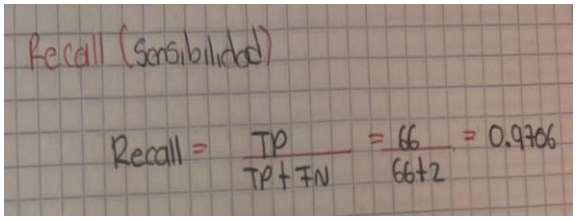


Imagen.22

El modelo alcanza un **recall del 97.06%**, lo que significa que logra detectar la gran mayoría de las fallas reales, dejando únicamente dos sin identificar.

- Costo Total del Error

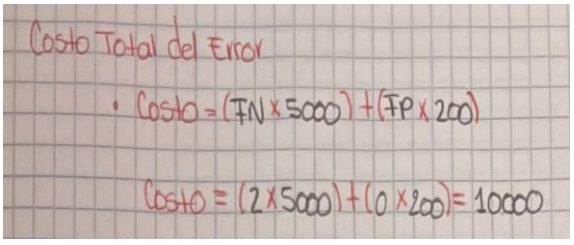


Imagen.23

El costo total del error es de **USD 10,000**, correspondiente a las fallas no detectadas. Este valor resalta la importancia de minimizar los falsos negativos en este tipo de problemas.

Preguntas y Reflexión.

1. **Trade-off entre Recall y Precisión: En el contexto de evitar paradas de la crítica, ¿preferiría usted un modelo con un Recall del 98% pero una Precisión del 60%, o viceversa? Justifique su respuesta basándose en los costos calculados anteriormente.**

En este contexto, es preferible priorizar un modelo con alto recall, incluso si esto implica una menor precisión. Esto se debe a que el costo de una falla no detectada **(USD 5,000)** es significativamente mayor que el de una falsa alarma **(USD 200)**.

Por lo tanto, resulta más conveniente aceptar algunas inspecciones innecesarias que permitir que una falla ocurra sin ser detectada, lo cual podría generar mayores pérdidas económicas.

2. **Curva de Aprendizaje: Observe la gráfica de los (pérdida) y accuracy durante el entrenamiento. ¿Considera que 30 ´épocas son suficientes para que el modelo converja? ¿Qué sucede con el error de validación comparado con el de entrenamiento**

A partir de los resultados del entrenamiento, se observa que el modelo mejora rápidamente durante las primeras épocas, reduciendo la pérdida y aumentando las métricas de desempeño. Tanto el accuracy como el recall alcanzan valores elevados, y las métricas de validación muestran un comportamiento consistente.

Esto indica que 30 épocas son suficientes para lograr la convergencia del modelo, e incluso se evidencia que el aprendizaje se estabiliza antes, lo que sugiere un entrenamiento eficiente sin señales claras de sobreajuste.

3. **Conclusión del Taller: Tras recorrer desde la regresión analitica hasta la clasificación con redes neuronales, ¿cómo puede el uso de métricas avanzadas (como el Recall) transformar la rentabilidad de una empresa de manufactura**

La red neuronal evaluada presenta un desempeño sobresaliente en la detección de fallas, alcanzando una alta precisión y un recall elevado. El uso de pesos de clase permitió mejorar la identificación de la clase minoritaria, lo cual es fundamental en problemas desbalanceados.

Desde una perspectiva de ingeniería, este modelo contribuye a la reducción de fallas no detectadas y a la optimización de la toma de decisiones, generando un impacto positivo en la eficiencia operativa y la rentabilidad del sistema productivo.

CONCLUSIONES

- A lo largo del desarrollo del taller se evidenció la importancia de los modelos matemáticos en la toma de decisiones dentro de sistemas productivos. Desde la regresión Ridge hasta las redes neuronales, se demostró cómo el uso de técnicas de machine learning permite mejorar la precisión en la predicción y clasificación de datos, aportando valor en contextos reales de ingeniería.

- La regularización Ridge permitió comprender cómo es posible controlar problemas como la multicolinealidad y el sobreajuste, garantizando estabilidad en los modelos mediante la inclusión del parámetro λ. Esto resalta la importancia de equilibrar la complejidad del modelo para obtener resultados más robustos y generalizables.

- El análisis del descenso de gradiente evidenció que la correcta elección de la tasa de aprendizaje es fundamental para lograr una convergencia eficiente. Se observó que valores muy altos generan inestabilidad, mientras que valores muy bajos ralentizan el proceso, lo que demuestra la necesidad de ajustar adecuadamente los hiperparámetros en modelos de optimización.

REFEENCIAS

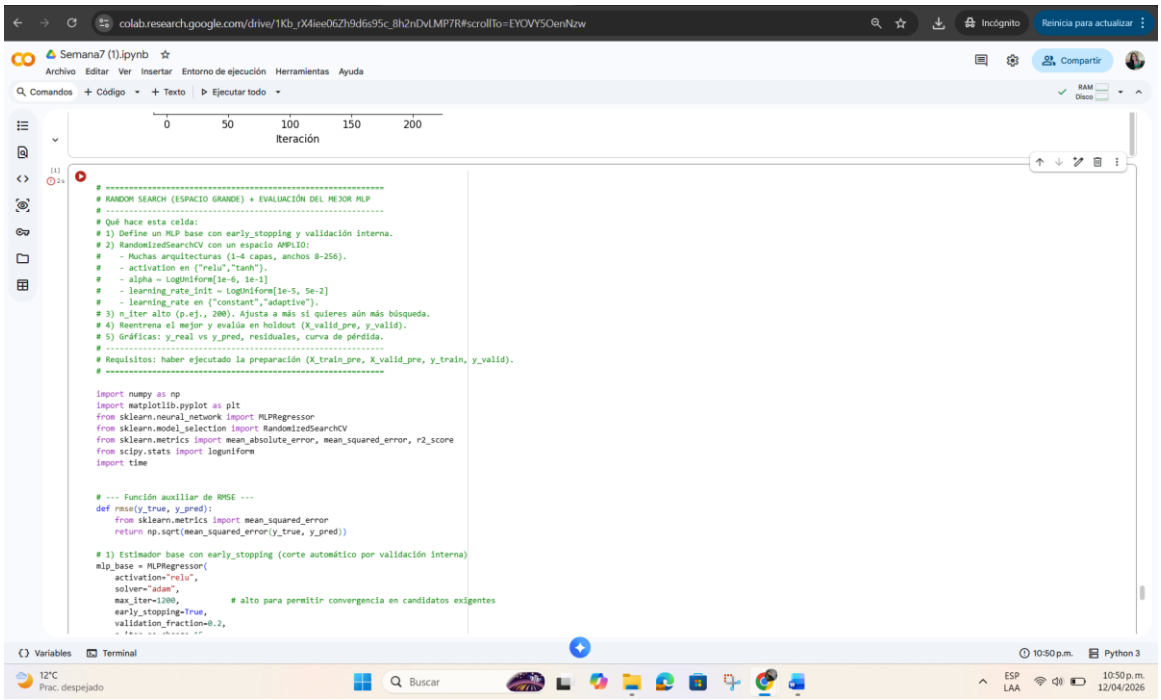
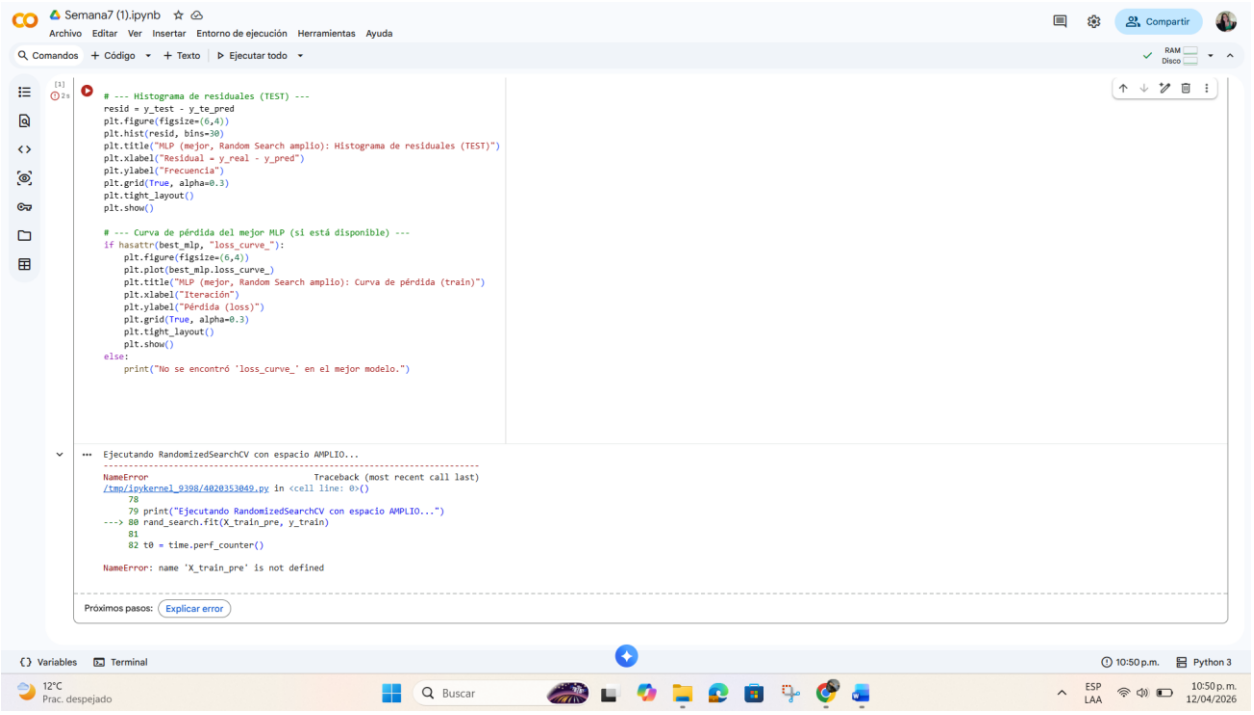
- Géron, A. (2020). Aprendizaje automático con Scikit-Learn, Keras y TensorFlow (2.ª ed.). O'Reilly Media.

- Morales, R. (2019). Introducción al aprendizaje automático: conceptos básicos y aplicaciones. Alfaomega.

- Pérez, C., & Santín, D. (2017). Machine learning: conceptos clave y aplicaciones prácticas. RA-MA Editorial.

- Hernández, J., & García, L. (2018). Redes neuronales artificiales: fundamentos y aplicaciones. Editorial Paraninfo.

- Sánchez, J. (2016). Minería de datos: técnicas y herramientas. Alfaomega.

- Domínguez, M. (2021). Inteligencia artificial aplicada a la ingeniería. Editorial UOC.

FALLA EN EL CODIGO SEMANA 7